

# EMPLOYEE LEAVE MANAGEMENT DATABASE SCHEMA

Constraint, Data Type, Relationship and Future Scaling Rationale

Design goal: keep transactional data consistent today while leaving clean extension points for future HR, attendance, payroll and workflow features.

## 1. Executive Summary

The schema has **six core entities**:

- **Department** – stores organizational departments.
- **Employee** – stores employees and manager hierarchy.
- **Leave Type** – defines available leave types.
- **Leave Balance** – tracks yearly leave allowance and usage.
- **Leave Request** – stores employee leave requests.
- **Approval History** – records approval/rejection actions.

### Core Design Principle

- **Primary Key** → unique identity of each record.
- **Foreign Key** → maintains relationships between tables.
- **UNIQUE** → prevents duplicate values.
- **CHECK** → allows only valid values.
- **NOT NULL** → ensures required data is provided.
- **DEFAULT** → provides common automatic values.

## 2. Relationship between table

| Relationship                      | Cardinality                  | Why it exists                                                                                                                |
|-----------------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Department -> Employee            | 1 to many                    | A department can contain zero or many employees; every employee belongs to one department because department_id is NOT NULL. |
| Employee -> Employee (manager_id) | 1 to many, optional on child | One manager can manage many employees. manager_id is nullable because top-level employees may have no manager.               |

| Relationship                      | Cardinality | Why it exists                                                                                 |
|-----------------------------------|-------------|-----------------------------------------------------------------------------------------------|
| Employee -> Leave Request         | 1 to many   | An employee can submit many requests; each request belongs to one employee.                   |
| Leave Type -> Leave Request       | 1 to many   | A leave type can be selected by many requests; each request chooses one leave type.           |
| Employee -> Leave Balance         | 1 to many   | One employee can have multiple balances, normally one per leave type and year.                |
| Leave Type -> Leave Balance       | 1 to many   | One leave type can have many employee-year balances.                                          |
| Leave Request -> Approval History | 1 to many   | A request can accumulate multiple workflow events such as manager approval and HR approval.   |
| Employee -> Approval History      | 1 to many   | An employee can act as approver in many history rows; each approval row records one approver. |

### 3. Cardinality in Simple Words

1. Department has many employees, but one employee belongs to one department.
2. An employee may have zero or one manager; a manager may have many direct reports.
3. An employee can submit many leave requests; each request belongs to one employee.
4. A leave type can be used by many requests and can appear in many annual balances.
5. A request can have many approval history records because approval can happen at multiple levels.
6. An approver can act on many requests, so approval\_history points back to Employee

### 4. Constraint and Datatype Used

| Term              | Meaning                      | Why it matters                                                                                         |
|-------------------|------------------------------|--------------------------------------------------------------------------------------------------------|
| PK                | Primary Key                  | Uniquely identifies one row and gives every entity a stable reference target.                          |
| FK                | Foreign Key                  | Ensures a referenced parent row exists, preserving referential integrity.                              |
| UK                | Unique Key                   | Stops duplicate values where the business expects uniqueness, such as email or leave type name.        |
| NOT NULL          | Value is required            | Prevents incomplete rows for attributes required by the workflow.                                      |
| DEFAULT           | Automatic value when omitted | Reduces repetitive application code and makes common states consistent.                                |
| CHECK             | Allowed values/rules         | Prevents invalid status, role, date or numeric values at the database layer.                           |
| ON DELETE CASCADE | Delete children with parent  | Useful where the child row has no meaning without the parent, such as balances for a deleted employee. |

## 5. DEPARTMENT Table

#### Columns, data types and constraints

| Column          | Datatype     | Constraint(s)    | Why this choice                                                                                                                                                   |
|-----------------|--------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| department_id   | SERIAL       | PRIMARY KEY      | Generates a simple integer identifier for each department. It is compact and efficient for joins and foreign keys.                                                |
| department_name | VARCHAR(100) | UNIQUE, NOT NULL | Names are human-readable text, 100 characters is generous for normal department names, and uniqueness prevents two departments having the exact same stored name. |

| Column          | Datatype          | Constraint(s) | Why this choice                                                                                                         |
|-----------------|-------------------|---------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>location</b> | VARCHAR(100)<br>) | Nullable      | A department may have a location, but a value may be unavailable or the department may operate remotely/multiple sites. |

### Future extension

If the organization has multiple offices, the location column can later be normalized into a Location table and Employee/Department can reference location\_id. The existing department\_id relationship can remain unchanged.

## 6. EMPLOYEE Table

### Columns, data types and constraints

| Column        | Datatype     | Constraint(s)                 | Why this choice                                                                                                                    |
|---------------|--------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| employee_id   | SERIAL       | PRIMARY KEY                   | Stable numeric identifier used by all employee-related foreign keys.                                                               |
| first_name    | VARCHAR(50)  | NOT NULL                      | Text is appropriate for names; required because employee identity needs at least a first name.                                     |
| last_name     | VARCHAR(50)  | Nullable                      | Some people or systems may not provide a last name, so the model does not force one.                                               |
| email         | VARCHAR(100) | UNIQUE, NOT NULL              | Text is required for email format; uniqueness prevents two employees sharing the same stored login/contact email.                  |
| phone         | VARCHAR(15)  | UNIQUE, Nullable              | Phone numbers should be stored as text because they may include leading zeros; nullable supports employees without a phone number. |
| department_id | INT          | NOT NULL, FK -> department    | Integer matches the parent key; every employee is required to belong to a department.                                              |
| manager_id    | INT          | Nullable, self-FK -> employee | Stores the reporting manager. Nullable supports top-level employees and avoids inventing a fake manager row.                       |
| role          | VARCHAR(20)  | NOT NULL, DEFAULT, CHECK      | A text code is simple to read; the CHECK restricts it to EMPLOYEE/MANAGER/HR.                                                      |
| joining_date  | DATE         | NOT NULL                      | Only a calendar date is needed for employment start; time-of-day is not necessary.                                                 |
| status        | VARCHAR(20)  | NOT NULL, DEFAULT, CHECK      | Stores lifecycle state and restricts it to ACTIVE or INACTIVE.                                                                     |

### Why manager\_id Matters

manager\_id creates a **self-referencing relationship** in the Employee table.

- It links an employee to their **manager**.
- One manager can have **many employees**.
- `NULL` allows top-level employees to have **no manager**.
- It represents the **organization hierarchy** without needing a separate Manager table.

### Constraint rationale

| Constraint             | Why it exists                                                                                            |
|------------------------|----------------------------------------------------------------------------------------------------------|
| fk_employee_department | Prevents an employee from referencing a department that does not exist.                                  |
| fk_employee_manager    | Prevents orphan manager references and allows the reporting hierarchy to be queried using recursive SQL. |
| chk_employee_role      | Stops typos such as "MANGER" from creating invalid authorization/workflow states.                        |
| chk_employee_status    | Keeps lifecycle values predictable for filtering active/inactive employees.                              |
| DEFAULT role/status    | Allows inserts to omit common values and still get consistent initial state.                             |

## 7. LEAVE\_TYPE Table

| Column          | Datatype     | Constraint(s)       | Why this choice                                                                                          |
|-----------------|--------------|---------------------|----------------------------------------------------------------------------------------------------------|
| leave_type_id   | SERIAL       | PRIMARY KEY         | Stable identifier referenced by requests and balances.                                                   |
| leave_type_name | VARCHAR(50)  | UNIQUE, NOT NULL    | Short human-readable name; unique so the catalog has one row per leave type name.                        |
| description     | VARCHAR(255) | Nullable            | Optional explanatory text; does not need an unlimited TEXT field for a compact description.              |
| max_days        | INT          | NOT NULL, CHECK > 0 | Whole-day maximum is modeled as integer policy data; the check prevents impossible zero/negative limits. |

## 7. LEAVE\_BALANCE Table

| Column         | Datatype     | Constraint(s)                   | Why this choice                                                                                                           |
|----------------|--------------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| balance_id     | SERIAL       | PRIMARY KEY                     | Stable row identity for the balance record.                                                                               |
| employee_id    | INT          | NOT NULL, FK, ON DELETE CASCADE | Associates the balance with its employee; cascade removes dependent balances if the employee master row is removed.       |
| leave_type_id  | INT          | NOT NULL, FK, ON DELETE CASCADE | Associates the balance with the leave policy; cascade keeps orphan balances from remaining after a leave type is deleted. |
| year           | INT          | NOT NULL, UNIQUE composite      | Separates annual entitlements and supports reporting by calendar year.                                                    |
| total_days     | NUMERIC(5,2) | NOT NULL, CHECK >= 0            | Decimal storage allows half-days or fractional entitlements without floating-point rounding issues.                       |
| used_days      | NUMERIC(5,2) | NOT NULL, DEFAULT 0, CHECK >= 0 | Usage starts at zero and can support half-days.                                                                           |
| remaining_days | NUMERIC(5,2) | NOT NULL, CHECK >= 0            | Stores the current available balance as a fast-read value.                                                                |

## 8. LEAVE\_REQUEST Table

| Column           | Datatype     | Constraint(s)                 | Why this choice                                                            |
|------------------|--------------|-------------------------------|----------------------------------------------------------------------------|
| leave_request_id | SERIAL       | PRIMARY KEY                   | Stable identifier for the transaction and for approval history.            |
| employee_id      | INT          | NOT NULL, FK -> employee      | Every request must belong to the person requesting leave.                  |
| leave_type_id    | INT          | NOT NULL, FK -> leave_type    | Every request must use a defined leave type.                               |
| start_date       | DATE         | NOT NULL                      | Leave start is calendar-based.                                             |
| end_date         | DATE         | NOT NULL, CHECK >= start_date | Prevents impossible ranges where an end date is before the start date.     |
| total_days       | NUMERIC(5,2) | NOT NULL, CHECK > 0           | Supports partial/half-day values while blocking zero or negative requests. |
| reason           | VARCHAR(500) | Nullable                      | Optional user-provided explanation with a moderate maximum size.           |

| Column     | Datatype    | Constraint(s)             | Why this choice                                                                         |
|------------|-------------|---------------------------|-----------------------------------------------------------------------------------------|
| status     | VARCHAR(30) | NOT NULL, DEFAULT, CHECK  | Captures workflow state and restricts it to the known lifecycle states.                 |
| created_at | TIMESTAMP   | DEFAULT CURRENT_TIMESTAMP | Records when the request row was created for audit/reporting; time-of-day matters here. |

## 9. APPROVAL\_HISTORY Table

| Column           | Datatype     | Constraint(s)                   | Why this choice                                                                                    |
|------------------|--------------|---------------------------------|----------------------------------------------------------------------------------------------------|
| approval_id      | SERIAL       | PRIMARY KEY                     | Stable identifier for each approval event.                                                         |
| leave_request_id | INT          | NOT NULL, FK, ON DELETE CASCADE | Every history row must belong to one request; cascade removes history when the request is deleted. |
| approver_id      | INT          | NOT NULL, FK -> employee        | Records the real employee who took the action.                                                     |
| approval_level   | VARCHAR(20)  | NOT NULL, CHECK                 | Restricts level to MANAGER or HR in the current two-step workflow.                                 |
| action           | VARCHAR(20)  | NOT NULL, CHECK                 | Restricts actions to APPROVED or REJECTED, making the event meaning explicit.                      |
| comments         | VARCHAR(500) | Nullable                        | Optional human explanation from the approver.                                                      |
| action_date      | TIMESTAMP    | DEFAULT CURRENT_TIMESTAMP       | Captures the exact event time for audit/history ordering.                                          |

### Why Approval History Is Separate

Approval History stores the **complete sequence of approval/rejection events** for a leave request.

- One leave request can have **multiple approval events**.
- Leave Request stores the **current status**.
- Approval History stores **who approved/rejected, when, and what happened**.
- Therefore, it has a **one-to-many relationship** with Leave Request.

#### Audit benefit

If the system stored only the latest status, it would be difficult to answer "Who approved it?", "When was it approved?" or "Was it first rejected and later resubmitted?" The history table preserves that timeline.

## 13. How to Scale the Schema in Future

The current six-table design is a solid transactional core. Scaling should mean adding entities around the core rather than putting unrelated concepts into the existing tables. New modules should usually introduce their own primary keys and reference the existing Employee, Leave Request, Department or Leave Type entities through foreign keys.

### Future entities

| Future entity | Example fields | Why add it |
|---------------|----------------|------------|
|---------------|----------------|------------|

| Future entity                            | Example fields                                                                          | Why add it                                                                                                                                      |
|------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Location</b>                          | office_id, name, address, timezone                                                      | Supports multi-office or multi-country organizations; can replace free-text Department.location when location becomes a reusable master entity. |
| <b>Holiday Calendar / Holiday</b>        | calendar_id, holiday_id, date, name                                                     | Lets the system calculate business days and exclude organization holidays.                                                                      |
| <b>Leave Policy</b>                      | policy_id, leave_type_id, department_id/role, accrual_rule, carry_forward_rule          | Moves policy logic out of Leave Type when rules differ by department, role, location or employee group.                                         |
| <b>Leave Balance Adjustment</b>          | adjustment_id, balance_id, amount, reason, created_by, created_at                       | Makes manual credits/debits auditable instead of directly editing balances.                                                                     |
| <b>Approval Workflow / Approval Step</b> | workflow_id, step_id, role/approver rule, sequence                                      | Supports more than the current MANAGER + HR two-step approval process.                                                                          |
| <b>Attendance</b>                        | attendance_id, employee_id, work_date, check_in, check_out, status                      | Connects leave with attendance and enables absence reconciliation.                                                                              |
| <b>Employee Role / Permission</b>        | role_id, permission_id, employee_id or role mapping                                     | Separates job role from application authorization when security needs become more granular.                                                     |
| <b>Notification</b>                      | notification_id, employee_id, leave_request_id, channel, status, sent_at                | Tracks email/SMS/in-app notifications without adding notification columns to Leave Request.                                                     |
| <b>Audit Log</b>                         | audit_id, actor_id, entity_type, entity_id, action, before_data, after_data, created_at | Provides broader system-level auditing beyond approval decisions.                                                                               |
| <b>Document / Attachment</b>             | document_id, employee_id, leave_request_id, file_url, file_type, uploaded_at            | Supports medical certificates or other supporting documents without bloating transaction tables.                                                |